

SDN & Network Automation

FIG_000 · guide v1.0 · 2026 · open reference · 9 sections

by Swastik Sagar

How networks stopped being configured one device at a time – the control/data plane split, programmable APIs, Infrastructure as Code, and what actually goes wrong when automation is done carelessly.

```
$ read section_01 --start
```

WHAT'S INSIDE

SECTION	TOPIC
1 – 3	What SDN is, controllers/southbound APIs, and northbound programmability
4 – 6	Infrastructure as Code, plus worked Ansible and Terraform examples
7 – 9	Intent-based networking, automation's benefits & risks, and reference

How to use this guide: read start to finish for the full picture, or jump to section 5 or 6 for the worked automation examples. Every diagram is numbered and referenced directly in the surrounding text.

Table of Contents

1. What Is SDN & the Control/Data Plane Split	3
2. SDN Controllers & Southbound APIs	4
3. Northbound APIs & Network Programmability	5
4. Infrastructure as Code for Networks	6
5. Worked Example: Ansible for Switch Configuration	7
6. Worked Example: Terraform for Cloud Networking	8
7. Intent-Based Networking	9
8. Benefits & Risks of Automation	10
9. Quick Reference & Glossary	11

This guide is designed to be read section by section, with diagrams positioned next to the concepts they illustrate.

What Is SDN & the Control/Data Plane Split

1.1 TWO PLANES, TRADITIONALLY BUNDLED TOGETHER

Every network device does two conceptually distinct jobs: the **control plane** decides *where* traffic should go – running routing protocols, building forwarding tables, making policy decisions – while the **data plane** actually *moves* the traffic, forwarding packets based on whatever the control plane decided. In a traditional network, both planes run together, independently, inside every single switch and router – each device makes its own decisions and forwards its own traffic, with no centralized view.

1.2 SDN: SEPARATING THE TWO

Software-Defined Networking (SDN) pulls the control plane out of individual devices entirely, centralizing it in a **controller** (Section 2) that has visibility into the whole network and makes forwarding decisions on behalf of every device. The devices themselves become comparatively simple – they just forward traffic exactly as instructed, with essentially none of the decision-making intelligence they'd otherwise run independently.



1.3 WHY CENTRALIZE AT ALL?

- **Whole-network visibility** – the controller sees the entire topology at once, enabling globally optimal decisions rather than each device reasoning only about its own local view.
- **Faster, consistent changes** – a policy update happens once, at the controller, instead of being pushed device-by-device.
- **Vendor abstraction** – applications talk to the controller's API (Section 3), not each device's individual, often vendor-specific configuration interface.

SDN Controllers & Southbound APIs

2.1 THE CONTROLLER'S JOB

The **SDN controller** is the centralized control plane from Section 1 – it maintains a real-time view of the network topology, computes forwarding decisions, and pushes those decisions down to the actual switches. Popular implementations include OpenDaylight, ONOS, and various vendor-specific controllers.

2.2 OPENFLOW: THE ORIGINAL SOUTHBOUND PROTOCOL

The **southbound interface** is how the controller communicates *down* to the physical/virtual switches it manages – the most historically significant southbound protocol is **OpenFlow**, which lets a controller directly install specific flow rules (match conditions and corresponding actions) into a switch's forwarding table. A switch receiving an unmatched packet can query the controller for instructions, or simply forward according to rules already installed.

CONCEPT	ROLE
Flow Table	The switch's local table of match-condition → action rules, populated by the controller
Flow Rule	A specific match (e.g., destination IP) paired with an action (e.g., forward out port 3)
Southbound API	The protocol/interface the controller uses to program switches (e.g., OpenFlow)

OpenFlow's influence outlasted its dominance. While OpenFlow itself is less universally deployed today than it once looked poised to become, the architectural pattern it established – a centralized controller programming simplified, standardized switches via an open protocol – fundamentally shaped how modern SDN and cloud networking (see the Cloud Networking guide) are architected, even where the specific protocol in use has moved on.

Northbound APIs & Network Programmability

3.1 THE OTHER DIRECTION

Where the southbound interface (Section 2) connects the controller *down* to switches, the **northbound API** connects it *up* to applications, orchestration tools, and human operators – typically a REST API returning and accepting structured data (usually JSON). This is what actually makes SDN *programmable*: instead of an engineer manually configuring devices one at a time, software can query network state and push changes through ordinary API calls.

3.2 WHAT BECOMES POSSIBLE

- **Self-service provisioning** – an application can request its own network resources (a new VLAN, a firewall rule) through an API call, without a human manually configuring a switch.
- **Integration with other systems** – the network can react programmatically to events from monitoring, orchestration, or ticketing systems, rather than requiring a human to notice and manually respond.
- **Custom applications on top of the network** – traffic engineering, security policy enforcement, or load-aware routing logic built as applications that consume the controller's northbound API, rather than being hardcoded into every device's firmware.

This is the foundation cloud networking is built on. The VPCs, security groups, and route tables from the Cloud Networking guide are themselves a form of SDN – when you call a cloud provider's API to create a subnet, you're using a northbound API, talking to a controller that programs the provider's underlying (southbound) switching infrastructure on your behalf, exactly matching this section's pattern.

3.3 SOUTHBOUND VS NORTHBOUND: QUICK COMPARISON

ASPECT	SOUTHBOUND API	NORTHBOUND API
Direction	Controller → switches	Applications/operators → controller
Typical protocol	OpenFlow, NETCONF, gRPC	REST/JSON
Purpose	Program forwarding behavior on devices	Expose network capability to software

Infrastructure as Code for Networks

4.1 CONFIGURATION AS VERSION-CONTROLLED TEXT

Infrastructure as Code (IaC) means defining network configuration – VLANs, routes, firewall rules, entire cloud VPCs – as text files, stored and versioned in the same kind of system (like Git) developers use for application source code, rather than as manual, one-off changes made directly on devices through a CLI or web console.

4.2 WHY THIS MATTERS BEYOND CONVENIENCE

- **Repeatability** – the exact same configuration can be deployed identically to a dev, staging, and production environment, eliminating "it worked when I set it up manually last time" drift.
- **Auditability** – every change has a commit history: who changed what, when, and (via commit messages/pull requests) often why.
- **Review before impact** – changes can be reviewed by a colleague before being applied, the same code-review discipline already standard for application software.
- **Disaster recovery** – if a device fails catastrophically, its entire configuration can be redeployed from the stored code, rather than reconstructed from memory or incomplete documentation.

4.3 DECLARATIVE VS IMPERATIVE

Most modern IaC tools are **declarative** – you describe the desired end state ("this VPC should have these three subnets"), and the tool figures out what changes are actually needed to get there, rather than requiring you to script the exact step-by-step sequence of commands (an **imperative** approach) yourself. This is why re-running a declarative tool against infrastructure that already matches the desired state safely does nothing – it only acts on the difference between current and desired state.

TOOL	PRIMARY DOMAIN	STYLE
Terraform	Cloud infrastructure, multi-provider	Declarative
Ansible	Configuration management, including on-prem devices	Primarily declarative, with imperative flexibility
CloudFormation	AWS-specific infrastructure	Declarative

Worked Example: Ansible for Switch Configuration

A simplified Ansible playbook configuring VLANs (see the VLANs & Trunking guide) consistently across multiple switches – the on-premises, device-level side of Infrastructure as Code (Section 4).

5.1 THE INVENTORY: WHICH DEVICES

```
$ [switches]
$ switch-a ansible_host=10.0.0.11
$ switch-b ansible_host=10.0.0.12
```

Defines which devices this playbook applies to – Ansible connects to each in turn (or in parallel) and applies the same configuration.

5.2 THE PLAYBOOK: DESIRED STATE

```
$ - name: Configure standard VLANs
$ hosts: switches
$ tasks:
$   - name: Ensure VLAN 10 exists
$     ios_vlan:
$       vlan_id: 10
$       name: Engineering
$       state: present
```

Declaratively states the desired end result – "VLAN 10, named Engineering, should exist" (Section 4.3) – rather than scripting the exact CLI commands to create it. Ansible determines whether the VLAN already exists and only makes changes if needed.

5.3 RUNNING THE PLAYBOOK

```
$ ansible-playbook -i inventory.ini vlans.yml
```

This applies the same, exact configuration to every switch in the inventory – switch-a and switch-b end up with identical VLAN 10 configuration, with zero risk of the manual typos or inconsistencies that come from configuring each device by hand, one at a time.

Worked Example: Terraform for Cloud Networking

Reusing the VPC-building example from the Cloud Networking guide, Section 8 – this time viewed through the Infrastructure as Code lens from Section 4 of this guide.

6.1 DECLARATIVE CLOUD NETWORKING

```
$ resource "aws_vpc" "main" {  
$   cidr_block = "10.0.0.0/16"  
$ }
```

Exactly like the Ansible example (Section 5.2), this declares a desired end state – "a VPC with this CIDR block should exist" – not a sequence of API calls. Terraform compares this declared state against what actually exists in the cloud provider, and computes the minimal set of changes needed.

6.2 THE PLAN-THEN-APPLY WORKFLOW

```
$ terraform plan  
$ terraform apply
```

`terraform plan` shows exactly what would change *before* anything actually happens – a preview that gets reviewed (Section 4.2's "review before impact" principle) before committing to `terraform apply`, which then makes the real changes. This two-step workflow is one of Terraform's most valuable safety properties: nothing changes silently or unexpectedly.

6.3 STATE: TERRAFORM'S MEMORY

Unlike Ansible, which typically re-queries actual device state on every run, Terraform maintains its own **state file** – a record of what it believes currently exists, used to compute the difference against your declared configuration. This makes Terraform very fast at detecting drift, but also means the state file itself becomes critical infrastructure – losing it means Terraform loses track of what it's actually managing.

Ansible vs Terraform isn't really a competition. They're commonly used together: Terraform provisioning the cloud infrastructure itself (VPCs, subnets, instances), and Ansible then configuring the software running inside those instances once they exist – each tool applied to the layer it's actually best suited for.

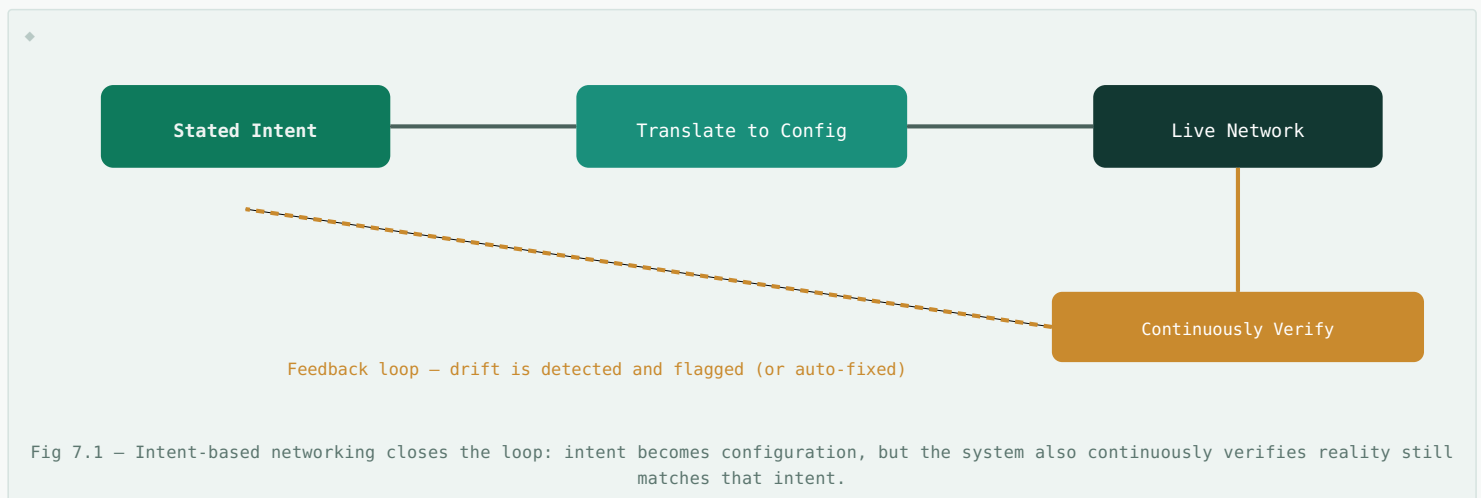
Intent-Based Networking

7.1 ONE LAYER ABOVE DECLARATIVE CONFIGURATION

Section 4's declarative IaC describes desired *configuration* – specific VLANs, specific routes. **Intent-Based Networking (IBN)** goes a level higher: you describe desired *business intent* ("the finance department's traffic must be isolated from the guest network, with a maximum of 50ms latency to the primary datacenter"), and the system translates that intent into the actual low-level configuration needed across every relevant device – then continuously verifies the network actually satisfies that intent, not just that the configuration was applied once.

7.2 CONTINUOUS VERIFICATION, NOT JUST DEPLOYMENT

The defining feature that separates IBN from ordinary IaC (Section 4) is the **closed feedback loop**: an intent-based system doesn't just push configuration and assume success – it continuously monitors actual network state against the stated intent, and can automatically detect (and sometimes automatically remediate) drift or violations, closing the loop between what you asked for and what's actually happening on the network in real time.



7.3 WHERE THIS FITS

IBN is a natural extension of everything earlier in this guide – it needs SDN's centralized visibility (Section 1) to actually observe the whole network, programmable APIs (Section 3) to both push configuration and pull real-time state, and IaC's declarative philosophy (Section 4) as its foundation, layered with the added continuous-verification loop that ordinary IaC deployment doesn't provide on its own.

Benefits & Risks of Automation

8.1 THE REAL BENEFITS

- **Speed** – changes that took hours of manual, device-by-device configuration happen in minutes or seconds.
- **Consistency** – every device gets the exact same configuration, eliminating the "works on switch A but not switch B" class of bugs entirely.
- **Reduced human error** – no typos in a hand-typed CLI command at 2am during an incident.
- **Scale** – managing thousands of devices by hand simply isn't feasible; automation is often not just faster but the *only* practical option at real scale.

8.2 THE REAL RISKS

- **Automated mistakes scale too.** A bad manual change affects one device; a bad automated change can affect every device simultaneously, in seconds – the exact same consistency that's automation's biggest strength becomes its biggest risk when the pushed configuration itself is wrong.
- **Reduced hands-on familiarity.** Engineers who mostly interact with the network through automation can lose the practical, hands-on troubleshooting instincts (Network Troubleshooting guide) that come from direct device-level work.
- **Tooling becomes critical infrastructure itself.** If the automation platform, its state file (Section 6.3), or its credentials are compromised or lost, that's now a serious incident in its own right – the automation layer needs the same security and resilience thinking as the network it manages.
- **Blind trust in "it applied successfully."** A deployment reporting success doesn't guarantee the network is actually behaving correctly – this is exactly the gap intent-based networking's continuous verification (Section 7.2) is designed to close.

The mitigation is the same discipline that makes automation valuable in the first place: the plan-before-apply workflow (Section 6.2), peer review before changes are applied (Section 4.2), staged rollouts rather than pushing to every device simultaneously, and genuine automated testing – treating network changes with the same rigor as application software deployment, not less.

Quick Reference & Glossary

TERM	DEFINITION
SDN	Software-Defined Networking – centralizing the control plane away from individual devices.
Control Plane	The part of a network device that decides where traffic should go.
Data Plane	The part of a network device that actually forwards traffic.
SDN Controller	The centralized system making forwarding decisions on behalf of managed switches.
Southbound API	The interface a controller uses to program switches (e.g., OpenFlow).
Northbound API	The interface applications/operators use to interact with a controller (typically REST).
OpenFlow	An early, influential protocol letting a controller install flow rules directly into switches.
Infrastructure as Code (IaC)	Defining infrastructure configuration as version-controlled text rather than manual changes.
Declarative	Describing a desired end state and letting a tool determine how to achieve it.
Imperative	Explicitly scripting the exact sequence of steps to reach a result.
Terraform	A popular declarative IaC tool, commonly used for cloud infrastructure.
Ansible	A configuration management tool, commonly used for on-prem device and server configuration.
Intent-Based Networking (IBN)	Describing desired business intent and continuously verifying the network satisfies it.
State File	Terraform's record of what it believes currently exists, used to compute needed changes.

End of guide. The throughline across this entire guide: separate the decision from the execution (SDN), describe the desired result instead of the steps to get there (IaC), and never stop checking that reality still matches what you asked for (IBN) – the same discipline, applied at increasing levels of abstraction.